

Web Technologies



WET2VO

Teil 3: PHP-Grundlagen Teil 2 und
Regular Expressions

Medientechnik und -design | SS 2026

Funktionen

Wiederverwendbare Code-Blöcke



Definieren &

Funktionsdeklaration

Aufruf

Funktionsname

```
function quadrat($x)
```

```
{
```

Parameter(optional, mehrere möglich)

```
    return $x * $x;
```

```
}
```

Wert zurückgeben (optional)

Aufruf:

```
$result = quadrat(5); // 25
```



Adv. Escaping

```
<?php
function noresult()
{
?>
<p>Die Abfrage lieferte kein Ergebnis!</p>
<?php
}
?>
```



Call by Value

Definition:

```
function cbv($x)
{
    return $x += 5;
}
```

← Eingangsparameter (Wert kopiert)

Aufruf:

```
$test1 = 5;
$result1 = cbv($test1);
echo $result1; // 10
echo $test1; // 5
```

← Ursprünglicher Wert unverändert



Call by Reference

Definition:

```
function cbr(&$x)
{
    return $x += 5;
}
```

Referenzparameter mit &-Zeichen
Nur bei der Funktionsdefinition.
Nicht beim Aufruf!

Aufruf:

```
$test2 = 5;
$result2 = cbr($test2);
echo $result2; // 10
echo $test2; // 10
```

Ursprünglicher Wert wurde auch geändert



Default Parameter

Definition:

```
function default($a, $b = "b", $c = "c")  
{  
    echo "<p>$a$b$c</p>";  
}
```

Aufruf:

```
default("a"); // abc  
default("a", "x"); // axc  
default("a", "x", "o"); // axo
```



First-Class Callable

Syntax

Definition inline function

```
function quadrat($x)  
{  
  return $x * $x;  
}
```

Funktion referenzieren, ohne sie auszuführen (einer Variable zuweisen):

```
$funcRef = quadrat(...);
```

In JavaScript:

```
const funcRef = quadrat;  
funcRef(10);
```

Späterer Aufruf über die Referenz:

```
echo $funcRef(10); // 100
```


Anonyme & Arrow - Funct.



Anonyme Funktion:

```
$callback = function ($name) {  
    return "Hallo $name";  
};  
echo "<p>{$callback("Igor")}</p>"; // Hallo Igor
```

Arrow-Function:

```
$arrow = fn($name) => "Hallo $name";  
echo "<p>{$arrow("Igor")}</p>"; // Hallo Igor
```

Pipe-Operator



Verschachtelter Funktionsaufruf:

```
$res1 = mb_substr(mb_strtoupper(mb_trim($input)), 0, 5);  
echo "<p>$res1</p>";
```

Pipe-Operator (linearer Datenfluss):

```
$res2 = $input  
    |> mb_trim(...)  
    |> mb_strtoupper(...)  
    |> (fn($x) => mb_substr($x, 0, 5));  
echo "<p>$res2</p>"; // HÄLLÖ
```



Typdeklarationen

Definition:

```
declare(strict_types=1);  
function isInt(int $number)  
{  
    echo "<p>Eine Zahl: $number</p>";  
}
```

Aufruf:

```
isInt(3); // okay  
isInt("3"); // teilweise okay  
isInt("hallo"); // nicht okay
```



Rückgabewertsdekl.

Definition:

```
declare(strict_types=1);  
function getSum($a, $b): int  
{  
    return $a + $b;  
}
```

Aufruf:

```
getSum(3, 4); // okay  
getSum(3.0, 4.0); // teilweise okay  
getSum(3.0, 2.5); // teilweise okay mit Fehler
```



Nullable Types

Definition:

```
function isIntOrNull(?int $number)
{
    echo "<p>Eine Zahl: $number</p>";
}

function getSumOrNull($a, $b): ?int
{
    return ($a + $b) > 0 ? $a + $b : null;
}
```



Void & Never

Definition:

```
function getNothing(): void
{
    echo "<p>Hier wird nichts zurückgegeben.</p>";
    return; // Optional, hier besser gleich weglassen
}
```

```
function redirect(string $uri): never
{
    header("Location: $uri");
    exit();
}
```



Union Types

Definition:

```
function getSum(int|float $a, int|float $b): int|float
{
    return $a + $b;
}
```

Aufruf:

```
getSum(3, 4); // 7
getSum(3.0, 4.0); // 7.0
getSum(3.0, 2.5); // 5.5
getSum(3, 2.5); // 5.5
```



Lokale Variablen

Definition:

```
function fooLokal()  
{  
    $x = "Lokal";  
}
```

Aufruf:

```
$x = "Global";  
fooLokal();  
echo $x; // Global
```




Globale Variablen

Definition:

```
function fooGlobal()  
{  
    global $x;  
    $x = "Jetzt global";  
}
```

Aufruf:

```
$x = "Global";  
fooGlobal();  
echo $x; // Jetzt global
```



Variablen, die **in** einer Funktion definiert wurden, sind außerhalb dieser Funktion nicht zugreifbar.

Variablen, die **außerhalb** einer Funktion definiert wurden, sind standardmäßig **in** der Funktion ebenfalls **nicht** zugreifbar.

Statische Variablen



Definition:

```
function counter()  
{  
    static $count = 0;  
    return $count++;  
}
```

Aufruf:

```
for ($i = 0; $i < 5; $i++) {  
    echo counter(); // 01234  
}
```

Konstanten

Bezeichner für unveränderbare Werte

Benannte Konstanten



Anlegen und ausgeben mit define():

```
define("USERNAME", "John Doe");  
echo USERNAME; // John Doe
```

Vorsicht: echo Username; // Error ab PHP 8 (davor Warnung)

Überprüfen:

```
if (defined("USERNAME")) {  
    echo "<p>Welcome " . USERNAME . "!</p>";  
}
```

Benannte Konstanten



Anlegen und ausgeben mit const:

```
const USER = "Jane Doe";  
echo USER; // Jane Doe
```

Überprüfen und Verschachtelung:

```
if (!defined("F00")) {  
    define("F00", "bar");  
    // const F00 = "bar"; -> funktioniert hier nicht  
}
```

Vordefinierte Konstanten



Magic Constants:

```
echo __LINE__; // 2
```

```
echo __FILE__; // /var/www/html/index.php
```

Core Constants:

```
echo PHP_VERSION; // 8.5.5
```

```
echo PHP_OS; // Linux
```

Standard Constants:

```
echo M_PI; // 3.1415926535898
```

```
echo M_E; // 2.718281828459
```

Arrays

Kollektionen von Werten

Erzeugen von Arrays



Indiziert:

```
$a1 = []; // Leer
```

```
$a2 = ["a", "b", "c"]; // Indiziert
```

```
$a3 = [3, "Hallo", 5.1]; // Gemischte Datentypen
```

Assoziativ:

```
$a4 = [0 => "a", 1 => "b", 2 => "c"];
```

```
$a5 = [0 => "a", 3 => "b", 4 => "c"];
```

```
$a6 = ["Vorname" => "John", "Nachname" => "Doe", "Alter" => "34"];
```

```
$a7 = ["farbe" => "rot", "form" => "rund", 3];
```

Erzeugen von Arrays



Mehrzeilige Schreibweise (bessere Lesbarkeit):

```
$a8 = [  
    "multi",  
    "line",  
    "declaration",  
];
```

Zugriff auf Arrays



Definition (wie zuvor):

```
$a2 = ["a", "b", "c"];  
$a4 = [0 ⇒ "a", 1 ⇒ "b", 2 ⇒ "c"];  
$a6 = ["Vorname" ⇒ "John", "Nachname" ⇒ "Doe", "Alter" ⇒ "34"];  
$a7 = ["farbe" ⇒ "rot", "form" ⇒ "rund", 3];
```

Zugriff:

```
$val1 = $a2[1]; // b  
$val2 = $a4[0]; // a  
$val3 = $a6["Vorname"]; // John  
$val4 = $a7["farbe"]; // rot  
$val5 = $a7[0]; // 3
```

Erweitern von Arrays



Indizierte Arrays:

```
$arr1 = [7, 4, 3]; // 0, 1, 2 belegt  
$arr1[] = 14; // Landet an Stelle 3
```

Lücken und gemischte Arrays:

```
$arr1 = [7, 4, 3]; // 0, 1, 2 belegt  
$arr1[7] = 14; // Landet an Stelle 7  
$arr1["test"] = 5; // Stelle "test"
```

```
/var/www/html/.../arrays.php:25:  
array (size=5)  
  0 ⇒ int 7  
  1 ⇒ int 4  
  2 ⇒ int 3  
  7 ⇒ int 14  
  'test' ⇒ int 5
```



Durchlaufen von Arrays



Indizierte Arrays:

```
$array1 = [3, 14, 4, 9];  
foreach($array1 as $value) {  
    echo $value; // 31449  
}
```

Assoziative Arrays:

```
$array2 = ["eins" => 1, "zwei" => 2, "drei" => 3];  
foreach($array2 as $key => $value) {  
    echo "$key:$value"; // eins: 1zwei: 2drei: 3  
}
```

Funktionen der PHP-Bibliothek

Funktionen für Variablen und Arrays

Existenz von Variablen



```
$a = "hallo";
```

Überprüfen:

```
isset($a); // true  
isset($b); // false
```

Löschen:

```
unset($a);  
isset($a); // false
```

Typprüfung von Variablen



```
$a = "hallo";
```

```
$b = 34;
```

```
$c = "1234";
```

```
is_array($a); // false
```

```
is_bool($a); // false
```

```
is_float($b); // false
```

```
is_int($b); // true
```

```
is_null($c); // false
```

```
is_numeric($c); // true
```

```
is_string($c); // true
```


Variablen **ausgeben**



```
$var = ["test", "wert", "eintrag"];  
print_r($var);  
var_dump($var);
```

Ausgabe print_r:

```
Array ( [0] => test [1] => wert [2] => eintrag )
```

Ausgabe var_dump:

```
/var/www/html/wet2vo-examples/vo03/php-funktionen/variablen-funktionen.php:9:
```

```
array (size=3)  
  0 => string 'test' (length=4)  
  1 => string 'wert' (length=4)  
  2 => string 'eintrag' (length=7)
```

Größe eines Arrays



Definition:

```
$arr1 = [2, "Hallo", 4.3];  
$arr2 = ["Test", 7, [3, 5, "Hi"]];
```

Ausgabe der Länge:

```
echo count($arr1); // 3  
echo count($arr2); // 3  
echo count($arr2, COUNT_RECURSIVE); // 6
```

Arrays durchsuchen



```
$arr3 = [1, 3, 5, 7];  
$arr4 = ["name" => "John Doe", "alter" => 34,  
        "geschlecht" => "männlich"];
```

Werte suchen und zurückgeben:

```
in_array(3, $arr3); // true  
array_first($arr3); // 1  
array_last($arr3); // 7
```

Schlüssel suchen und zurückgeben:

```
array_key_exists("alter", $arr4); // true  
array_key_first($arr4); // name  
array_key_last($arr4); // geschlecht
```

Arrays zu String & retour



String in Array aufteilen:

```
$text = "Hello little world!";  
$pieces = explode(" ", $text);  
print_r($pieces); // Array ( [0] => Hello [1] => little [2] => world! )
```

Array zu String zusammenfügen:

```
$parts = ["name", "alter", "geschlecht"];  
$string = implode(",", $parts);  
echo $string; // name,alter,geschlecht
```

Verarbeiten von Formulardaten

Einführung in die serverseitige Formularverarbeitung

3 superglobale Arrays



3 superglobale Arrays:

`$_GET` - `$_POST` - `$_REQUEST`

HTML-Formular:

Alternativ: `post`

```
<form method="get" action="get.php">
  <label for="info" >Info:</label>
  <input type="text" name="info" id="info" >
  <button type="submit">Abschicken</button>
</form>
```

Zugriff in PHP nach Absenden des Formulars:

```
$text = $_GET["info"]; ← Hier dann: $_POST["info"]
```

Mehrfachauswahl

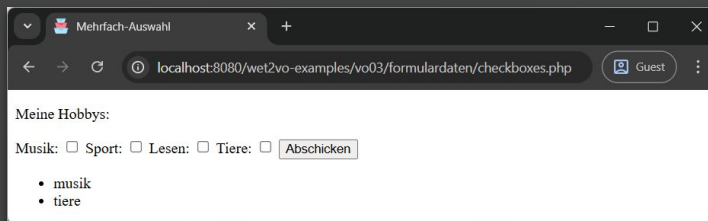


HTML-Formular:

```
<form method="post" action="checkboxes.php">
  <input type="checkbox" name="hobbies[]"
    value="musik" id="music">
  <input type="checkbox" name="hobbies[]"
    value="sport" id="sports">
  <input type="checkbox" name="hobbies[]"
    value="lesen" id="reading">
  <input type="checkbox" name="hobbies[]"
    value="tiere" id="animals">
  <button type="submit">Abschicken</button>
</form>
```

Zugriff in PHP:

```
if (isset($_POST["hobbies"])) {
    echo "<ul>";
    foreach($_POST["hobbies"] as $hobby) {
        echo "<li>$hobby</li>";
    }
    echo "</ul>";
}
```



Regular Expressions

Zeichenmengen und Muster allgemein beschreiben

Warum Regex?

Beispiel Datum: 23.04.2026

Oder 2026-04-23, oder 04/23/2026, ...





Regex

Überprüfen, ob ein Muster im String vorhanden ist:

```
preg_match($pattern, $subject [, $matches]);
```

Alle Vorkommen des Musters finden:

```
preg_match_all($pattern, $subject [, &$matches, $flags])
```

Ein Muster mit anderem Muster ersetzen:

```
preg_replace($pattern, $replacement, $subject);
```



Regex

Bei Muster einen Callback aufrufen:

Funktionen

```
preg_replace_callback($pattern, $callback, $subject);
```

Bei mehreren Mustern entsprechende Callbacks aufrufen:

```
preg_replace_callback_array($pattern_cb_array, $subject);
```



Einfache Muster

`" / . . . / "`

Schema: Anführungszeichen und Delimiter

Einfache Beispiele:

`" / a / "` (enthält ein "a")

`" / test / "` (enthält die Zeichenfolge "test")

Anwendung mit Funktion:

```
preg_match("/a/", "Hallo Welt!"); // 1 (true)
```

Wildcard



" / . aus / "

Stellvertretend für alle Zeichen

Erkennt u.a.:

Maus, Haus, raus

Punkt-Zeichen wird escaped!

" / \ . aus / "



Zeichenklassen

Zeichenklassen:

`"/[abc ]/"` (entweder a, b, c oder Leerzeichen)

`"/[a-z]/"` (Kleinbuchstaben von a bis z)

`"/[0-5]/"` (Zahlen von 0 bis 5)

`"/[a-zA-Z0-9_]/"` (Klein- und Großbuchstaben, Zahlen, Underscore)

`preg_match("/[abc ]/", "Hallo Welt!");` // 1 (true)

Vordefinierte Klassen:

`"/[[:digit:]]/"`, `"/[[:alpha:]]/"`, `"/[[:alnum:]]/"`,

`"/[[:lower:]]/"`, ...

Negierte Klassen (alles außer diesen Zeichen):

`"/[^aeiou]/"`



Anfang und Ende

Beginnt mit:

`"/^Die/`

`"/^[0-9]/`

Endet mit:

`"/er$/`

Beginnt und endet mit (genau dieser String):

`"/^ABC$/`



Häufigkeiten

?: 0 oder 1

{m}: Genau m Mal

+: 1 bis n

{m, n}: Zwischen m und n Mal

*****: 0 bis n

{m, }: Mindestens m Mal

{, n}: Maximal n Mal

Beispiele:

```
preg_match("/ca+t/", "caaaaaaat"); // 1 (true)
```

```
preg_match("/ca+t/", "ct"); // 0 (false)
```

```
preg_match("/ca?t/", "caaaaaaat"); // 0
```

```
preg_match("/ca*t/", "ct"); // 1
```

```
preg_match("/[0-9]{3}-[0-9]{3}-[0-9]{4}/", "123-456-7890"); // 1
```


Gruppierung / Subpattern



SGruppierung:

```
preg_match("/a(bc)*/", "aaaaa"); // 1 (true)
preg_match("/a(bc)*/", "abc"); // 1
preg_match("/a(bc)*/", "bc"); // 0 (false)
preg_match("/a(bc)*/", "ab"); // 0
```

Gruppierung mit \$matches-Array:

```
if (preg_match("/([0-9]{1,2})\.([0-9]{1,2})\.([0-9]{4})/",
    "23.04.2026", $matches)) {
    echo $matches[0]; // 23.04.2026
    echo $matches[1]; // 23
    echo $matches[2]; // 04
    echo $matches[3]; // 2026
}
```



Alternativen

Alternativmöglichkeiten:

```
preg_match("/cat|dog/", "The cat bit me"); // 1 (true)  
preg_match("/cat|dog/", "The dog bit me"); // 1 (true)  
preg_match("/cat|dog/", "The rabbit bit me"); // 0 (false)
```

Vorsicht bei Gruppierung:

`"/^cat|dog$/"` (beginnt mit cat oder endet mit dog)

vs.

`"/^(cat|dog)$/"` (muss entweder mit cat oder dog beginnen und enden)



Escape -Sequenzen

Nicht druckbare Zeichen:

`\n`: Ein Zeilenumbruch

`\t`: Tabulator

Zeichen-Typen:

`\d`: Jede beliebige Dezimalzahl

`\D`: Jedes Zeichen, das keine Dezimalzahl ist

`\s`: Jede Art von Whitespace-Zeichen (Zeichen-Typ).

`\S`: Jedes Zeichen, das kein Whitespace ist (Zeichen-Typ).

Bedingungen:

`\b`: Eine Wortgrenze

`\B`: Keine Wortgrenze



Nächstes Mal in WEF2VO

Objektorientiertes PHP

Credits

- Keine Referenzen in diesem Foliensatz.

